REAL ESTATE RECOMMENDATION SYSTEM

Recommendation Systems — Course Project Report

Student Name	Class	Roll Number	Semester
Atul Kumar	EXCS(A)	23108A0064	VII

1. Abstract

This project presents an end-to-end real-estate recommendation system inspired by the RE-RecSys research architecture. It processes public property listings from Mumbai, Gurgaon, Hyderabad, and Kolkata and generates ranked recommendations using explicit preferences and simulated interaction histories. The processed catalog contains 38,482 properties; the experimental framework generates 1,200 synthetic users and 15,559 simulated interactions. Users are classified as cold-start, short-term, long-term, or short-long-term and routed to rule-based, content-based, collaborative, or hybrid recommenders. Content-based filtering uses weighted user profiles and cosine similarity, while collaborative filtering uses Alternating Least Squares (ALS). A FastAPI backend and Streamlit interface provide explainable recommendations and interaction controls. In the stored offline evaluation, the rule-based model achieved the strongest measured ranking metrics. Because the interactions are synthetic and generated using preference-matching logic related to the rule-based score, the results describe this experimental setting rather than real customer behaviour.

2. Introduction

Real-estate platforms contain large and heterogeneous catalogs. Listings differ in city, locality, price, property type, BHK count, area, furnishing, amenities, and transaction type. Manually browsing a large inventory can be time-consuming and may return many irrelevant properties. Recommendation systems address this problem by ranking items according to a user's stated preferences and observed behaviour.

A central challenge is the cold-start problem: a new user has little or no interaction history, so collaborative filtering has limited evidence. Users with richer histories can benefit from content similarity and patterns learned from other users. This project uses interaction-based user categorization and model routing to select a recommendation strategy according to available information. The design is inspired by RE-RecSys [1], but is an independent academic implementation using public listings and simulated interactions.

2.1 Objectives

- Integrate and clean property listings collected from multiple city-specific schemas.
- Generate reproducible synthetic user profiles and simulated user–property interactions.
- Implement rule-based, content-based, collaborative, and hybrid recommendation methods.
- Classify users by interaction history and route them to an appropriate recommender.
- Provide ranked, explainable recommendations through an API and interactive interface.
- Evaluate ranking quality using standard offline recommendation metrics.

2.2 Problem Statement

Given a property catalog and a user profile containing explicit preferences and, where available, interaction history, the system must return a ranked top-K list of eligible properties. The strategy should adapt to the amount of information available about the user and avoid using held-out interactions during model training.

2.3 Relevance to Recommendation Systems

Concept	Project application	Boundary
Data integration	Harmonizes four city datasets into a canonical property schema.	Local file-based processing.
Data preparation	Parses prices, area, BHK, amenities and categories; handles duplicates and invalid values.	Some source facets remain incomplete.
Feature engineering	Builds structured/text feature representations and sparse user–property matrices.	Single-machine execution.
Analytical evaluation	Computes ranking metrics and produces comparison plots/tables.	Offline evaluation only.

The implementation does not use Hadoop, Spark, Kafka, distributed model training, or cloud infrastructure. Its Big Data relevance lies in data integration, feature processing, sparse interaction analysis, model evaluation, and visualization rather than distributed-scale deployment.

3. System Analysis & Design

3.1 Dataset and Software Requirements

The raw property data contains listings rather than genuine user clickstreams. The project therefore generates synthetic user profiles and interactions for controlled offline experiments.

Table 1. Dataset summary

Item	Verified value / description
Source cities	Mumbai, Gurgaon, Hyderabad, Kolkata
Raw property records	38,502
Processed property records	38,482
Synthetic users	1,200
Simulated interactions	15,559
Training interactions	10,888
Test interactions	4,671
Interaction types	Impression, image view, detail view, amenity view, shortlist, lead, conversion
Main output	Ranked properties with scores and explanations

Table 2. Main software requirements

Technology	Purpose
Python	Core implementation
Pandas, NumPy	Data preparation and numerical processing
Scikit-learn	TF-IDF, preprocessing, normalization and cosine similarity
SciPy	Sparse matrices
implicit	ALS collaborative filtering
PyYAML, Joblib	Configuration and model persistence
FastAPI, Uvicorn	REST API
Streamlit	Interactive user interface
Matplotlib, Pytest	Plots and automated tests

3.2 Methodology

The system reads raw city listing files, harmonizes their schemas, parses price and property attributes, removes duplicates, and creates a canonical catalog. It then generates synthetic user profiles and interaction events, builds property features, trains the ALS model, classifies users, routes requests to the appropriate recommendation method, ranks eligible properties, and returns explanations through the API or frontend.

3.2.1 Rule-Based Recommendation

The rule-based model is primarily used for cold-start users. It computes a weighted match score from explicit preferences such as budget, location, BHK, property type, furnishing, and amenities; configured cohort-popularity signals may also contribute. A simplified expression is:

$$S_r(p,u) = \Sigma f\ w_f\ m_f(p,u) / \Sigma f\ w_f \quad (1)$$

Here, m_f(p,u) is the match for feature f between property p and user u, and w_f is the configured feature weight. This method is transparent and works without interaction history, but depends on manually selected rules and does not learn hidden cross-user patterns.

3.2.2 Content-Based Filtering

The content-based model represents properties using available categorical, numerical, amenity, and text attributes. It constructs a user profile as a weighted combination of feature vectors from the user's interaction history, giving stronger events such as shortlists or leads more influence than weak events. Candidate properties are ranked by cosine similarity:

$$sim(u,p) = (v_u \cdot x_p) / (||v_u||\ ||x_p||) \quad (2)$$

Here, v_u is the user's feature profile and x_p is the property vector. Previously interacted properties and ineligible candidates are filtered according to the implementation. Content-based filtering can respond to new interactions, but may repeatedly recommend properties similar to those already viewed.

3.2.3 Collaborative Filtering Using ALS

Collaborative filtering builds a sparse user–property matrix whose values represent weighted implicit interactions. Alternating Least Squares learns latent user and property factors that approximate the interaction structure:

$$R \approx XY^T \quad (3)$$

R is the interaction matrix, X contains user latent factors, and Y contains property latent factors. ALS can identify patterns not directly encoded in listing attributes, but its quality depends on interaction coverage and is limited for new users or items.

3.2.4 Hybrid Recommendation

The hybrid model combines normalized content and collaborative scores:

$$S_h(u,p) = \alpha S_c(u,p) + (1 - \alpha) S_c f(u,p) \quad (4)$$

S_c and $S_c f$ are the content and collaborative scores, respectively; α controls their relative contribution. The configured default is α = 0.5. The stored alpha sweep identified α = 0.9 as best in the tested simulated environment; this is not evidence that the same value is optimal for real users.

3.3 User Classification and Recommendation Routing

The routing logic adapts the model to the amount and recency of a user's interaction history. Because the generated data has no genuine event timestamps, recency is represented by an is_recent flag rather than real elapsed time.

Table 3. User categories and routing

Category	Implemented condition	Selected model
Cold-start	No interaction history / zero total interactions	Rule-based
Short-term	Fewer than 5 total interactions and at least 1 marked recent	Content-based
Long-term	At least 5 total interactions and no marked recent interactions	Hybrid
Short-long-term	At least 5 total interactions and at least 1 marked recent	Hybrid

The frontend stores live simulated interactions in session state, while the API maintains interactions in the in-memory orchestrator. New interactions can immediately affect content-based recommendations because the model reads the updated history. ALS is not retrained after every individual interaction.

3.4 Evaluation Methodology

For users with at least two interactions, events are ordered and the final 30% are held out for testing, while at least one interaction remains in training. The main model comparison evaluates up to 150 users at K = 5, 10, and 20. Training uses only the training portion; the evaluation is offline and measures recovery of held-out interactions.

Precision@K measures the fraction of recommended items that are relevant; Recall@K measures the fraction of held-out relevant items retrieved:

$$\textbf{Precision@K = relevant items in top K / K} \quad (5)$$

$$\textbf{Recall@K = relevant items in top K / all relevant test items} \quad (6)$$

Average Precision rewards relevant items near the top of the ranked list, and MAP is its mean across users. NDCG@K normalizes discounted cumulative gain by the ideal ranking, giving higher weight to relevant items near the top. Hit Rate@K is 1 when at least one relevant test item appears in the top K, and 0 otherwise.

Figure 1. End-to-end system architecture

Data preparation	User & interaction data	Recommendation and delivery
Raw city CSVs ↓ Schema harmonization ↓ Cleaning and parsing ↓ Canonical property catalog	Synthetic user profiles + Simulated interactions ↓ Feature engineering ↓ Weighted interaction matrix	User categorization ↓ Rule-based / Content-based / ALS collaborative / Hybrid ↓ Ranked explanations ↓ FastAPI + Streamlit

Figure 1. Data preparation, interaction modelling, adaptive recommendation, and application delivery.

Figure 2. Recommendation workflow

User preferences and history → user categorization → model selection → candidate filtering and ranking → explanation generation → recommendation cards → View / Shortlist / Lead event → updated interaction history.

Figure 2. The workflow supports interaction-driven updates; ALS is not retrained for each event.

4. Implementation

4.1 Data Preprocessing and Synthetic Data

The preprocessing pipeline converts four city-specific source files into a canonical property schema. It addresses inconsistent column names, price values expressed in lakh/crore or rent formats, ranges and text-based prices, nested location fields, facet-based categories, amenities, duplicate property IDs, and missing or invalid price, area, and BHK values. Feature engineering also applies transformations and outlier handling where configured. The resulting catalog contains 38,482 property records.

Since the source listings do not contain genuine users or browsing histories, the project generates 1,200 synthetic profiles and 15,559 simulated interactions. This makes model training and offline evaluation possible, but results remain dependent on the assumptions used to generate those interactions.

4.2 Recommendation Modules

Table 4. Principal source modules and responsibilities

Module	Responsibility
src/data/prepare.py	Ingests source files and creates the canonical property catalog.
src/users/generate_users.py	Creates synthetic user profiles.
src/interactions/generate_interactions.py	Generates simulated interaction histories.
src/features/property_features.py	Builds property feature representations.
src/models/rule_based.py	Preference matching and popularity-based ranking.
src/models/content_based.py	Weighted user profiles and cosine similarity.
src/models/collaborative.py	Sparse interaction matrix and ALS model.
src/models/hybrid.py	Combines content and collaborative scores.
src/users/categorize.py; src/recommendation/orchestrator.py	Classifies users, routes requests, filters candidates and creates explanations.
scripts/train.py; scripts/evaluate.py	Training, persistence and offline evaluation.

4.3 API and Frontend

The FastAPI backend exposes endpoints for health checks, property listing and detail retrieval, user creation and profile retrieval, interaction submission, and recommendation retrieval. The Streamlit frontend lets users select a demo profile or create a profile using city, transaction type, BHK, property type, furnishing, and budget. Recommendation cards display property details, scores, the selected method, and explanations, with View, Shortlist, and Lead actions. These actions create simulated interactions and refresh the interface.

4.4 Automated Testing

The project test suite was executed locally using `python -m pytest -q` from the project directory. The verified run reported 59 passed in 6.44 seconds. These tests assess software behaviour and are separate from recommendation-quality evaluation, which uses offline ranking metrics. The test count is specific to this execution and may change if the suite is modified.

5. Results and Discussion

5.1 Experimental Summary

Table 5. Evaluation population

Statistic	Value
Processed properties	38,482
Synthetic users	1,200
Simulated interactions	15,559
Training / test interactions	10,888 / 4,671
Users in main model comparison	150
Main comparison cut-offs	K = 5, 10, 20

5.2 Model Comparison

Table 6. Model comparison at K = 10 for 150 evaluated users

Model	Precision@10	Recall@10	MAP@10	NDCG@10	Hit Rate@10
Rule-based	0.0127	0.0177	0.0085	0.0199	0.1133
Content-based	0.0093	0.0135	0.0048	0.0126	0.0733
Collaborative	0.0027	0.0038	0.0007	0.0028	0.0267
Hybrid	0.0053	0.0072	0.0027	0.0072	0.0467

The rule-based model achieved the strongest measured values across the listed metrics. Content-based filtering ranked second, while the hybrid model improved on collaborative filtering but did not exceed the rule-based or content-based methods in this comparison. The absolute scores are low, so the table should be interpreted as a comparative offline result rather than evidence of high real-world recommendation accuracy.

5.3 Discussion and Limitations

The observed ranking is strongly affected by the synthetic-data design. The interaction generator uses preference-match logic related to the rule-based scoring function, giving that model a structural advantage when recovering held-out simulated events. Consequently, the experiment does not establish that rule-based recommendation is generally superior for real property seekers. The collaborative model also has limited evidence because the interaction matrix is sparse.

- The system uses simulated rather than real user behaviour and lacks genuine browsing timestamps.
- Offline held-out interactions do not measure satisfaction, enquiry conversion, or property purchases.
- The evaluation is performed on a limited user sample and a single-machine environment.
- Some source facets are incomplete, and no online A/B test or production-scale deployment is reported.
- The implementation does not use distributed Big Data infrastructure.

6. Conclusion & Future Scope

The project implements an end-to-end real-estate recommendation pipeline covering multi-source listing preparation, synthetic user and interaction generation, feature engineering, four recommendation approaches, user-category routing, explainable ranked results, offline evaluation, API delivery, and an interactive frontend. In the stored experiment, the rule-based model achieved the strongest measured ranking metrics, followed by content-based filtering. However, synthetic interaction generation introduces bias in favour of preference matching, so the results should not be generalized to real customers.

Future work should prioritize collecting anonymized real interaction data, adding genuine timestamps and session-aware evaluation, selecting hybrid weights on independent validation data, improving property facet coverage, incorporating geographic or image-based similarity, and evaluating the system through user studies or online experiments. Distributed processing may be considered if the dataset and operational requirements grow substantially.

7. References

[1] V. C. Venkatesh, H. Oberoi, A. Goyal, and N. Sikka, “RE-RecSys: An End-to-End System for Recommending Properties in Real-Estate Domain,” CODS-COMAD 2024, arXiv:2404.16553.

[2] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, “Recommender Systems Survey,” Knowledge-Based Systems, vol. 46, pp. 109–132, 2013.

[3] R. Chen, Q. Hua, Y.-S. Chang, B. Wang, L. Zhang, and X. Kong, “A Survey of Collaborative Filtering-Based Recommender Systems,” IEEE Access, vol. 8, pp. 4594–4623, 2020.

[4] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” Computer, vol. 42, no. 8, pp. 30–37, 2009.

[5] P. Lops, M. de Gemmis, and G. Semeraro, “Content-Based Recommender Systems: State of the Art and Trends,” in Recommender Systems Handbook, Springer, 2011.